

Name: Mohd Aadil

Class: TYIT

Subject: AI

UID: 24BIT054

Roll No.: 45

CA Project Submission

Dots & Boxes AI Game

1. Introduction

Dots & Boxes is a two-player strategy game where players take turns drawing lines between adjacent dots. Whenever a player completes the fourth side of a box, that player claims the box and receives an extra turn.

In this project, the human player competes against an AI opponent.

The project is developed using Python and Tkinter. The AI combines multiple decision-making techniques:

- Greedy capture
- DFS
- BFS
- Heuristic / risk evaluation
- *A-style safe-move ranking**
- Minimax-style Alpha-Beta search
- Randomization/fallback behavior is not actually implemented in the current code

The game's available board sizes are 3×3, 4×3 and 5×4, and the interface displays AI reasoning and search statistics.

2. Objectives

The main objectives are:

Technical objectives

- Develop an interactive Dots & Boxes game.
- Implement a graphical user interface using Tkinter.
- Allow the player to select different board sizes.
- Implement an AI opponent.
- Detect completed boxes automatically.
- Implement DFS-based chain detection.
- Implement BFS-based chain detection.

- Use a heuristic to identify safe and unsafe moves.
- Implement A*-style candidate ranking.
- Implement Alpha-Beta search for endgame sacrifice decisions.
- Track player and AI scores.
- Display AI reasoning to the user.

AI objectives

The major AI objective is not simply to find a move, but to select a move according to the current strategic situation.

The AI follows this priority:

1. Capture a free box



2. Detect chains



3. Find a safe move



4. If no safe move exists



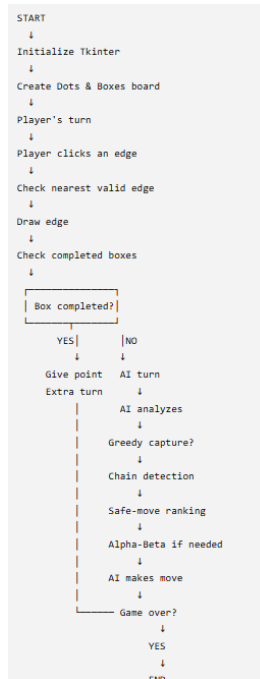
5. Use Alpha-Beta search

3. Technologies Used

Technology	Purpose
Python	Main programming language
Tkinter	GUI development
ttk	Combobox / styled widgets
Collections deque	BFS queue
DFS	Chain detection
BFS	Chain detection
Heuristic evaluation	Safe/unsafe move evaluation
A*-style ranking	Ranking safe candidates
Alpha-Beta	Endgame decision-making

4. Game Flow

The complete game flow is:

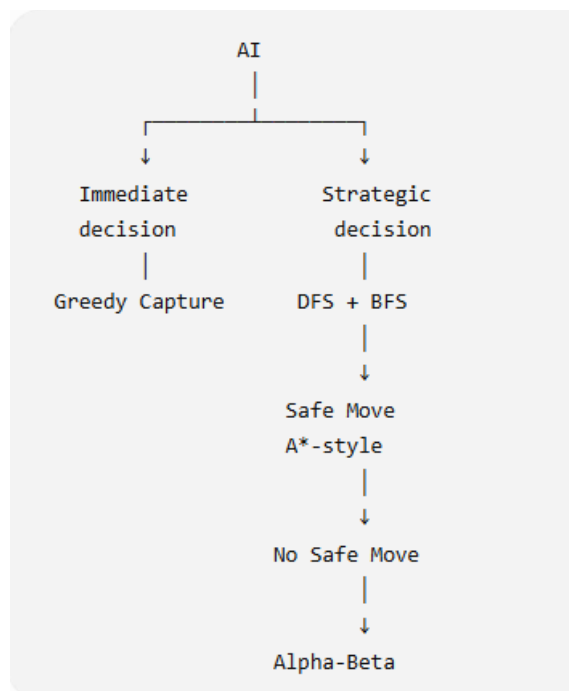


5. AI Algorithm Theory

Your AI is actually a hybrid AI.

Instead of depending on only one algorithm, it uses different techniques for different situations.

AI hierarchy



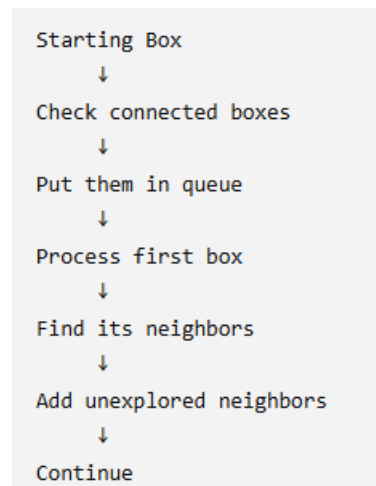
6. BFS

Definition

BFS = Breadth-First Search

BFS explores nodes level by level.

BFS working



7. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax.

Instead of evaluating every possible branch, it removes branches that cannot influence the final answer.

if $\beta \leq \alpha$: `ab_pruned += 1 break`

8. Alpha / Beta Values

Alpha

Alpha represents the best value found so far for the maximizing player.

`alpha = float("-inf")`

Beta

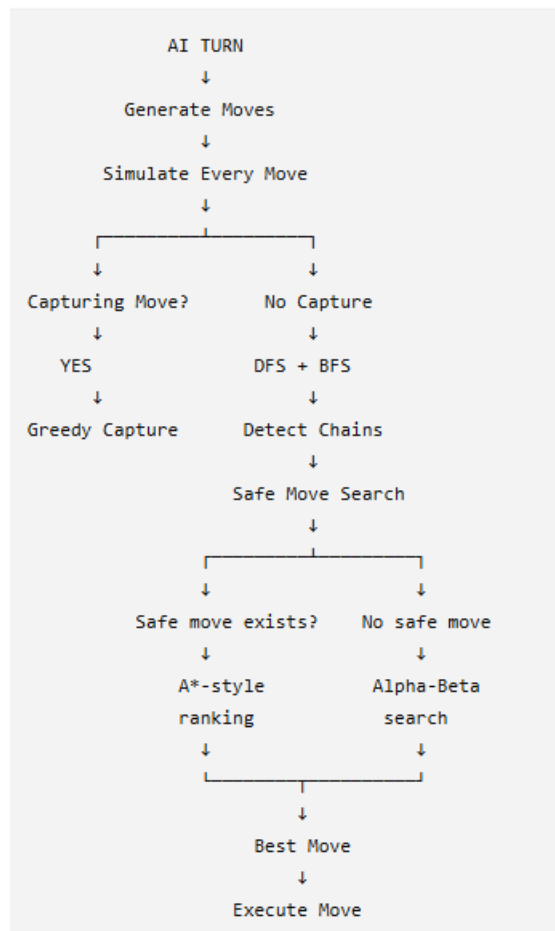
Beta represents the best value found so far for the minimizing player.

`beta = float("inf")`

search starts with:

`float("-inf"), float("inf")`

9. AI Decision-



Code:

```
Explorer
...
dots_and_boxes.py X
dots_and_boxes.py
Dots_And_Boxes
dots_and_boxes.py

dots_and_boxes.py > DotsBoxes > __init__
1 import tkinter as tk
2 from tkinter import ttk
3 from collections import deque
4
5 BG, PANEL, GRID, INK, DIM = "#0b1210", "#0f1a17", "#22362e", "#d7e4de", "#7f9c93"
6 GREEN, AMBER, BLUE, RED = "#4dff00", "#ffb84d", "#4dc4ff", "#ff5f6d"
7 SPACING, MARGIN, HIT_DIST = 80, 46, 14
8
9
10 class DotsBoxes:
11     def __init__(self, root):
12         self.root = root
13         root.title("Dots & Boxes // AI Lab")
14         root.configure(bg=BG)
15         self.bboxes_x = self.bboxes_y = 0
16         self.drawn, self.owners = set(), []
17         self.scores = {"human": 0, "ai": 0}
18         self.current_player, self.game_active, self.ai_thinking = "human", True, False
19         self.hover_edge = None
20         self.build_ui()
21         self.new_game()
22
23 # ----- UI -----
24 def build_ui(self):
25     tk.Label(self.root, text="DOTS & BOXES // AI LAB", bg=BG, fg=GREEN,
26             font=("Consolas", 15, "bold")).pack(pady=(18, 2))
27     tk.Label(self.root, bg=BG, fg=DIM, font=("Consolas", 10),
28             text="DFS + BFS find the chains  A* picks the safe move  "
29             "Alpha-Beta decides the sacrifice").pack(pady=(0, 18))
```

```
dots_and_boxes.py > DotsBoxes > __init__
10 class DotsBoxes:
24 def build_ui(self):
31
32     main = tk.Frame(self.root, bg=BG)
33     main.pack()
34     left = tk.Frame(main, bg=BG)
35     left.pack(side="left", padx=(0, 22), anchor="n")
36
37     self.canvas = tk.Canvas(left, bg=PANEL, highlightbackground=GRID,
38                             highlightthickness=1, cursor="hand2")
39     self.canvas.pack()
40     self.canvas.bind("<Motion>", self.on_hover)
41     self.canvas.bind("<Leave>", self.on_leave)
42     self.canvas.bind("<Button-1>", self.on_click)
43
44     self.status = tk.Label(left, text="", bg=BG, fg=GREEN, font=("Consolas", 11, "bold"))
45     self.status.pack(pady=(10, 0))
46
47     side = tk.Frame(main, bg=BG, width=300)
48     side.pack(side="left", anchor="n")
49
50     game = self.card(side, "GAME")
51     self.size_var = tk.StringVar(value="4x3")
52     self.size_select = ttk.Combobox(game, textvariable=self.size_var, state="readonly",
53                                     values=("3x3", "4x3", "5x4"), font=("Consolas", 10))
54     self.size_select.pack(fill="x", pady=(0, 8))
55     self.size_select.bind("<<ComboboxSelected>>", lambda e: self.new_game())
56     tk.Button(game, text="NEW GAME", command=self.new_game, bg="#0a1512", fg=AMBER,
57               activebackground="#0a1512", activeforeground=AMBER, relief="solid", bd=1,
58               font=("Consolas", 10)).pack(fill="x")
```

```

59     score = self.card(side, "SCORE")
60     self.you_label = self.stat(score, "You", BLUE)
61     self.ai_label = self.stat(score, "AI", RED)
62     self.left_label = self.stat(score, "Boxes left", INK)
63
64     reason = self.card(side, "AI REASONING - LAST MOVE")
65     self.reasoning = tk.Label(reason, text="", bg=PANEL, fg=GREEN, justify="left",
66                               wraplength=265, font=("Consolas", 9))
67     self.reasoning.pack(fill="x", pady=(0, 8))
68     self.dfs_label = self.stat(reason, "DFS nodes visited", INK)
69     self.bfs_label = self.stat(reason, "BFS nodes visited", INK)
70     self.astar_label = self.stat(reason, "A* candidates ranked", INK)
71     self.ab_label = self.stat(reason, "Alpha-Beta nodes / pruned", INK)
72
73     help_card = self.card(side, "HOW IT PLAYS")
74     help_text = ("Click near a dotted line to draw it.\n\n"
75                 "Complete the 4th side of a box to claim it and get an extra turn.\n\n"
76                 "The AI captures free boxes, uses DFS and BFS to detect chains, "
77                 "A*-style ranking for safe moves, and Alpha-Beta minimax when forced "
78                 "to sacrifice.")
79     tk.Label(help_card, text=help_text, bg=PANEL, fg=DIM, justify="left",
80             wraplength=265, font=("Consolas", 9), anchor="w").pack(fill="x")
81
82     def card(self, parent, title):
83         frame = tk.Frame(parent, bg=PANEL, bd=1, relief="solid", padx=14, pady=12)
84         frame.pack(fill="x", pady=(0, 12))
85         tk.Label(frame, text=title, bg=PANEL, fg=DIM,
86                 font=("Consolas", 9, "bold")).pack(anchor="w", pady=(0, 8))
87         return frame
88

```

```

89     def stat(self, parent, name, fg):
90         row = tk.Frame(parent, bg=PANEL)
91         row.pack(fill="x", pady=2)
92         tk.Label(row, text=name, bg=PANEL, fg=DIM, font=("Consolas", 9)).pack(side="left")
93         value = tk.Label(row, text="0", bg=PANEL, fg=fg, font=("Consolas", 9, "bold"))
94         value.pack(side="right")
95         return value
96
97     # ----- Board helpers -----
98     def all_edges(self):
99         edges = [("h", r, c) for r in range(self.bboxes_y + 1) for c in range(self.bboxes_x)]
100         edges += [("v", r, c) for r in range(self.bboxes_y) for c in range(self.bboxes_x + 1)]
101         return edges
102
103     def box_edges(self, r, c):
104         return [("h", r, c), ("h", r + 1, c), ("v", r, c), ("v", r, c + 1)]
105
106     def box_sides(self, r, c):
107         return sum(e in self.drawn for e in self.box_edges(r, c))
108
109     def box_missing_edges(self, r, c):
110         return [e for e in self.box_edges(r, c) if e not in self.drawn]
111
112     def adjacent_boxes(self, edge):
113         typ, row, col = edge
114         if typ == "h":
115             return [(row - 1, col)] * (row - 1 >= 0) + [(row, col)] * (row < self.bboxes_y)
116         return [(row, col - 1)] * (col - 1 >= 0) + [(row, col)] * (col < self.bboxes_x)
117
118     def owner_index(self, r, c):
119         return r * self.bboxes_x + c

```

```

120
121 def dot_pos(self, row, col):
122     return MARGIN + col * SPACING, MARGIN + row * SPACING
123
124 # ----- Game engine -----
125 def new_game(self):
126     self.bboxes_x, self.bboxes_y = map(int, self.size_var.get().split("x"))
127     self.drawn = set()
128     self.owners = [None] * (self.bboxes_x * self.bboxes_y)
129     self.scores = {"human": 0, "ai": 0}
130     self.current_player, self.game_active, self.ai_thinking = "human", True, False
131     self.hover_edge = None
132
133     w = self.bboxes_x * SPACING + MARGIN * 2
134     h = self.bboxes_y * SPACING + MARGIN * 2
135     self.canvas.config(width=w, height=h)
136
137     for lbl in (self.reasoning, self.dfs_label, self.bfs_label, self.astar_label, self.ab_label):
138         lbl.config(text="")
139     self.status.config(text="Your move – click a line to draw it.")
140     self.update_score_ui()
141     self.draw()
142
143 def apply_move(self, edge, player):
144     self.drawn.add(edge)
145     completed = []
146     for r, c in self.adjacent_boxes(edge):
147         idx = self.owner_index(r, c)
148         if self.owners[idx] is None and self.box_sides(r, c) == 4:
149             self.owners[idx] = player

```

```

148         if self.owners[idx] is None and self.box_sides(r, c) == 4:
149             self.owners[idx] = player
150             completed.append((r, c))
151     return completed
152
153 def boxes_remaining(self):
154     return sum(o is None for o in self.owners)
155
156 def update_score_ui(self):
157     self.you_label.config(text=str(self.scores["human"]))
158     self.ai_label.config(text=str(self.scores["ai"]))
159     self.left_label.config(text=str(self.boxes_remaining()))
160
161 def check_game_over(self):
162     if self.boxes_remaining() != 0:
163         return False
164     self.game_active = False
165     h, a = self.scores["human"], self.scores["ai"]
166     msg = "You win!" if h > a else "AI wins." if a > h else "It's a tie."
167     self.status.config(text=f"Game over – {msg} ({h} : {a})")
168     return True
169
170 # ----- Drawing -----
171 def draw(self):
172     self.canvas.delete("all")
173     for r in range(self.bboxes_y):
174         for c in range(self.bboxes_x):
175             owner = self.owners[self.owner_index(r, c)]
176             if not owner:
177                 continue

```

```

177         continue
178     x, y = self.dot_pos(r, c)
179     fill = "#16313d" if owner == "human" else "#3b2026"
180     tcol = BLUE if owner == "human" else RED
181     self.canvas.create_rectangle(x + 4, y + 4, x + SPACING - 4, y + SPACING - 4,
182                                fill=fill, outline="")
183     self.canvas.create_text(x + SPACING / 2, y + SPACING / 2,
184                            text="YOU" if owner == "human" else "AI",
185                            fill=tcol, font=("Consolas", 10, "bold"))
186
187     for edge in self.all_edges():
188         typ, row, col = edge
189         x1, y1 = self.dot_pos(row, col)
190         x2, y2 = self.dot_pos(row, col + 1) if typ == "h" else self.dot_pos(row + 1, col)
191         if edge in self.drawn:
192             self.canvas.create_line(x1, y1, x2, y2, fill=GREEN, width=4)
193         elif edge == self.hover_edge:
194             self.canvas.create_line(x1, y1, x2, y2, fill=AMBER, width=4)
195         else:
196             self.canvas.create_line(x1, y1, x2, y2, fill=GRID, width=2, dash=(2, 4))
197
198     for r in range(self.bboxes_y + 1):
199         for c in range(self.bboxes_x + 1):
200             x, y = self.dot_pos(r, c)
201             self.canvas.create_oval(x - 4, y - 4, x + 4, y + 4, fill=INK, outline="")
202

```

```

204     def point_to_segment_distance(self, px, py, x1, y1, x2, y2):
205         dx, dy = x2 - x1, y2 - y1
206         length_sq = dx * dx + dy * dy
207         t = max(0, min(1, ((px - x1) * dx + (py - y1) * dy) / length_sq)) if length_sq else 0
208         cx, cy = x1 + t * dx, y1 + t * dy
209         return ((px - cx) ** 2 + (py - cy) ** 2) ** 0.5
210
211     def find_nearest_edge(self, x, y):
212         best, best_dist = None, HIT_DIST
213         for edge in self.all_edges():
214             if edge in self.drawn:
215                 continue
216             typ, row, col = edge
217             x1, y1 = self.dot_pos(row, col)
218             x2, y2 = self.dot_pos(row, col + 1) if typ == "h" else self.dot_pos(row + 1, col)
219             d = self.point_to_segment_distance(x, y, x1, y1, x2, y2)
220             if d < best_dist:
221                 best_dist, best = d, edge
222         return best
223
224     def on_hover(self, event):
225         if not self.game_active or self.current_player != "human" or self.ai_thinking:
226             return
227         self.hover_edge = self.find_nearest_edge(event.x, event.y)
228         self.draw()
229
230     def on_leave(self, event):
231         self.hover_edge = None
232         self.draw()

```

```

233
234 def on_click(self, event):
235     if not self.game_active or self.current_player != "human" or self.ai_thinking:
236         return
237     edge = self.find_nearest_edge(event.x, event.y)
238     if edge is None:
239         return
240
241     completed = self.apply_move(edge, "human")
242     self.scores["human"] += len(completed)
243     self.hover_edge = None
244     self.update_score_ui()
245     self.draw()
246
247     if self.check_game_over():
248         return
249     if completed:
250         self.status.config(text="Box claimed – go again.")
251     else:
252         self.current_player = "ai"
253         self.status.config(text="AI is thinking...")
254         self.ai_thinking = True
255         self.root.after(400, self.ai_turn_loop)
256
257 # ----- DFS + BFS chain detection -----
258 def eligible_boxes(self):
259     return [(r, c) for r in range(self.bboxes_y) for c in range(self.bboxes_x)
260             if self.owners[self.owner_index(r, c)] is None and self.box_sides(r, c) == 2]
261
262 def linked_boxes(self, box):
263     r, c = box
264     links = []
265     for edge in self.box_missing_edges(r, c):

```

```

266         for nb in self.adjacent_boxes(edge):
267             if nb == box:
268                 continue
269             nr, nc = nb
270             if self.owners[self.owner_index(nr, nc)] is None and self.box_sides(nr, nc) == 2:
271                 links.append(nb)
272     return links
273
274 def find_chains_dfs(self):
275     visited, chains, nodes = set(), [], 0
276
277     def dfs(box, group):
278         nonlocal nodes
279         nodes += 1
280         visited.add(box)
281         group.append(box)
282         for nb in self.linked_boxes(box):
283             if nb not in visited:
284                 dfs(nb, group)
285
286     for box in self.eligible_boxes():
287         if box not in visited:
288             group = []
289             dfs(box, group)
290             chains.append(group)
291     return chains, nodes
292
293 def find_chains_bfs(self):
294     visited, chains, nodes = set(), [], 0
295     for start in self.eligible_boxes():
296         if start in visited:
297             continue

```

```

298         group, queue = [], deque([start])
299         visited.add(start)
300         while queue:
301             box = queue.popleft()
302             nodes += 1
303             group.append(box)
304             for nb in self.linked_boxes(box):
305                 if nb not in visited:
306                     visited.add(nb)
307                     queue.append(nb)
308             chains.append(group)
309         return chains, nodes
310
311     # ----- AI helpers -----
312     def simulate_move(self, edge):
313         boxes = self.adjacent_boxes(edge)
314         before = [self.box_sides(r, c) for r, c in boxes]
315         self.drawn.add(edge)
316         completes, creates_three, newly_eligible = [], False, 0
317
318         for i, (r, c) in enumerate(boxes):
319             after = self.box_sides(r, c)
320             if after == 4:
321                 completes.append((r, c))
322             elif after == 3:
323                 creates_three = True
324             elif after == 2 and before[i] == 1:
325                 newly_eligible += 1
326
327         self.drawn.remove(edge)
328         return {"completes": completes, "creates three": creates_three, "newly eligible": newly_eligible}

```

```

361 # 4. Alpha-Beta forced sacrifice
362 self.astar_label.config(text="0 (none safe)")
363 if not chains:
364     return moves[0], "Fallback move (no chains detected).".
365
366 ab_nodes = ab_pruned = 0
367
368 def eval_sacrifice(remaining, ai_score, human_score, opener_is_ai, depth, alpha, beta):
369     nonlocal ab_nodes, ab_pruned
370     ab_nodes += 1
371     if not remaining or depth == 0:
372         return ai_score - human_score
373
374     maximizing = opener_is_ai
375     best = float("-inf") if maximizing else float("inf")
376
377     for i, chain in enumerate(remaining):
378         rest = remaining[:i] + remaining[i + 1:]
379         gain = len(chain)
380         new_ai = ai_score + (0 if opener_is_ai else gain)
381         new_human = human_score + (gain if opener_is_ai else 0)
382         value = eval_sacrifice(rest, new_ai, new_human, not opener_is_ai,
383                               depth - 1, alpha, beta)
384         if maximizing:
385             best, alpha = max(best, value), max(alpha, max(best, value))
386         else:
387             best, beta = min(best, value), min(beta, min(best, value))
388         if beta <= alpha:
389             ab_pruned += 1
390             break
391     return best

```

OUTPUT:

DOTS & BOXES // AI LAB

DFS + BFS find the chains A* picks the safe move Alpha-Beta decides the sacrifice

Your move – click a line to draw it.

GAME

4x3

NEW GAME

SCORE

You

0

AI

0

Boxes left

12

AI REASONING – LAST MOVE

-

DFS nodes visited

-

BFS nodes visited

-

A* candidates ranked

-

Alpha-Beta nodes / pruned

-

HOW IT PLAYS

Click near a dotted line to draw it.

Complete the 4th side of a box to claim it and get an extra turn.

The AI captures free boxes, uses DFS and BFS to detect chains, A*-style ranking for safe moves, and Alpha-Beta minimax when forced to sacrifice.

GAME

4x3

3x3

4x3

5x4

Game over – AI wins. (1 : 11)

GAME

4x3

NEW GAME

SCORE

You

1

AI

11

Boxes left

0

AI REASONING – LAST MOVE

Greedy capture – free box available.

DFS nodes visited

6

BFS nodes visited

6

A* candidates ranked

0 (none safe)

Alpha-Beta nodes / pruned

15 / 2